Migrating from Monolith to Microservices

A Practical Guide for Backend Engineering Teams

Alex Chen
Principal Engineer, Platform Team
March 2026

The Monolith Today

Why we need to change

D

Deployment Bottleneck

- Single deployable = 4-hour release windows
- All 20 engineers blocked on one pipeline
- Average 2.3 deploys/week (target: daily)

B

Blast Radius

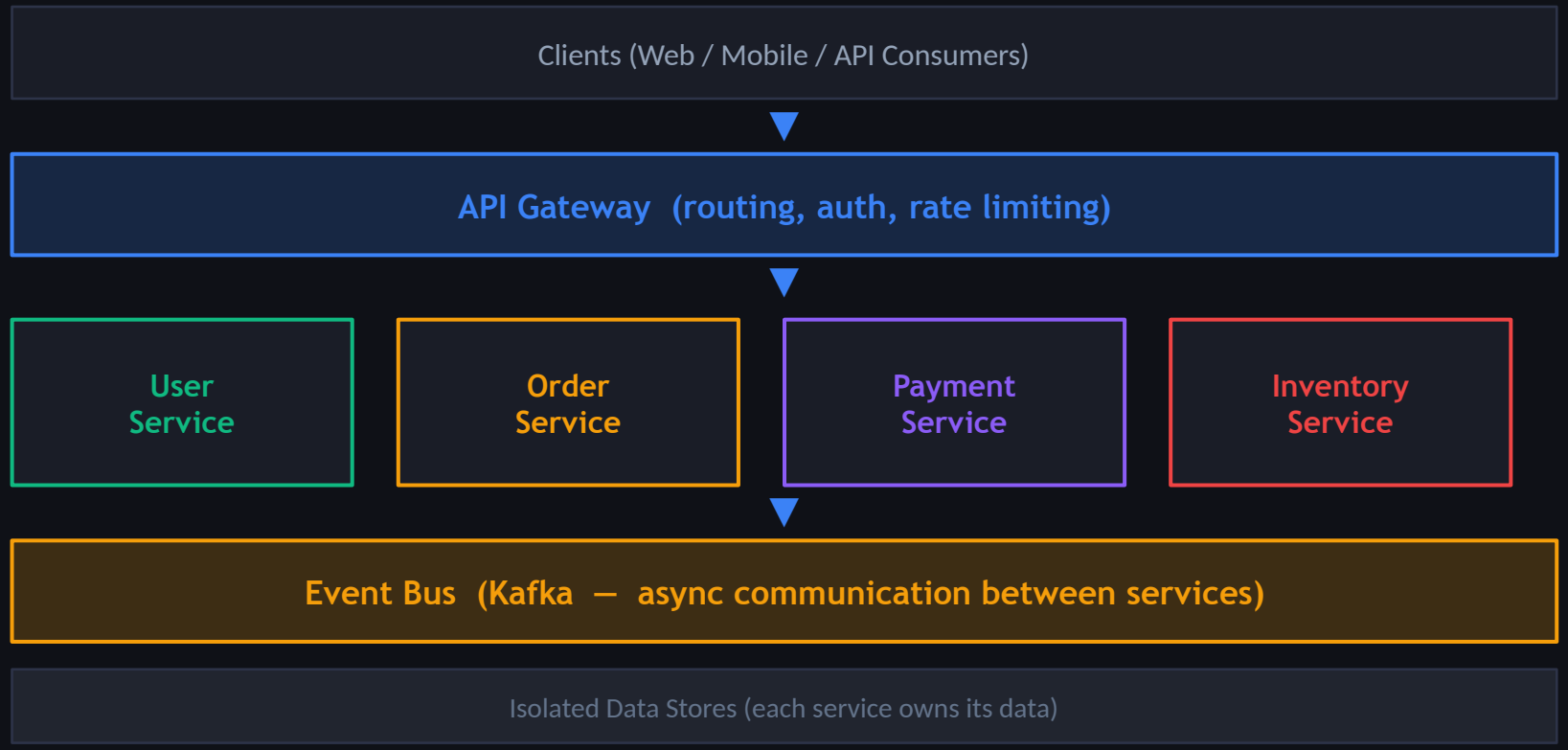
- One bad commit takes down everything
- No fault isolation between domains
- MTTR: 47 min avg (last quarter)

C

Team Coupling

- Merge conflicts across unrelated features
- Shared DB schema = hidden dependencies
- No clear ownership boundaries

Target Architecture



Migration Strategy: Strangler Fig

Incrementally replace monolith capabilities without a big-bang rewrite

01

Identify & Isolate

Map domain boundaries.
Extract shared DB tables.
Define service contracts.

02

Facade Layer

Insert API gateway.
Route traffic to monolith.
Add feature flags.

03

Extract Services

Peel off one service at a time. Dual-write for data consistency.

04

Retire Monolith

Redirect all traffic.
Decommission old code.
Clean up shared DB.

Risk Matrix

Risk	Likelihood	Impact	Mitigation
Data inconsistency during dual-write period	HIGH	HIGH	Event sourcing + reconciliation jobs running every 5 min
Service discovery failures in early rollout	MEDIUM	HIGH	Circuit breakers + fallback to monolith endpoints
Network latency increase from service-to-service calls	HIGH	MEDIUM	Async messaging via Kafka; cache hot paths
Team skill gap on distributed systems	MEDIUM	MEDIUM	Training program + embedded SRE in each squad
Operational complexity (20+ services to manage)	HIGH	LOW	Standardized templates, shared CI/CD pipeline

6-Month Phased Rollout

Month 1	Month 2	Month 3	Month 4	Month 5	Month 6
Foundation	**First Service**	**Expand**	**Accelerate**	**Stabilize**	**Complete**
API gateway setup Service mesh infra CI/CD templates	Extract User Service Dual-write migration Integration tests	Extract Order Service Event bus (Kafka) Monitoring baseline	Payment + Inventory Remove dual-writes Load testing	Performance tuning Chaos engineering Runbook creation	Monolith retirement DNS cutover Retrospective

Monitoring & Observability

Instrument from day one — you cannot debug what you cannot see

Metrics

- Request rate, error rate, duration (RED)
- Queue depth & consumer lag
- CPU / memory / GC per service
- Custom business metrics (orders/sec)

Traces

- Distributed tracing (OpenTelemetry)
- Trace ID propagation across services
- Latency breakdown per hop
- Automatic span instrumentation

Logs

- Structured JSON logging (no plaintext)
- Correlation IDs in every log line
- Centralized aggregation (ELK / Loki)
- Alert on error rate spikes

Tooling Stack

- **Prometheus + Grafana**
  Metrics & dashboards
- **Jaeger / Tempo**
  Distributed tracing
- **ELK Stack / Loki**
  Log aggregation
- **PagerDuty**
  Alerting & on-call
- **OpenTelemetry SDK**
  Unified instrumentation

Team Structure

Organize squads around service boundaries ("You build it, you run it")

User Squad	4 engineers	User Service, Auth, Profiles
Commerce Squad	5 engineers	Order Service, Cart, Checkout
Payments Squad	4 engineers	Payment Service, Billing, Refunds
Inventory Squad	3 engineers	Inventory Service, Warehouse API
Platform Squad	4 engineers	API Gateway, CI/CD, Infra, Observability

Key Principles

- Full ownership: build, deploy, operate
- On-call rotation per squad
- Cross-squad API contract reviews
- Weekly architecture sync
- Embedded SRE in Platform squad
- Shared runbook repository

Success Metrics

< 200ms

P99 Latency Target

Currently: 450ms

10x/day

Deploy Frequency

Currently: 2.3x/week

< 15min

MTTR Target

Currently: 47 min

Metric	Current	6-Month Target
Change failure rate	18%	< 5%
Lead time for changes	2 weeks	< 1 day
Service availability	99.5%	99.95%
Test coverage	42%	> 80%

Questions?

Let's build this together.

Alex Chen
alex.chen@company.dev | #platform-migration | Confluence: /microservices